

PHASE 22 — FINAL PRODUCTIZATION, SIH STRATEGY, EXECUTION ROADMAP, PITCH & HOW TO WIN

Project: Enterprise Cryptographic Discovery & Analysis Tool (ECDAT)

Problem Statement: SIH26164

Organization: National Technical Research Organisation (NTRO)

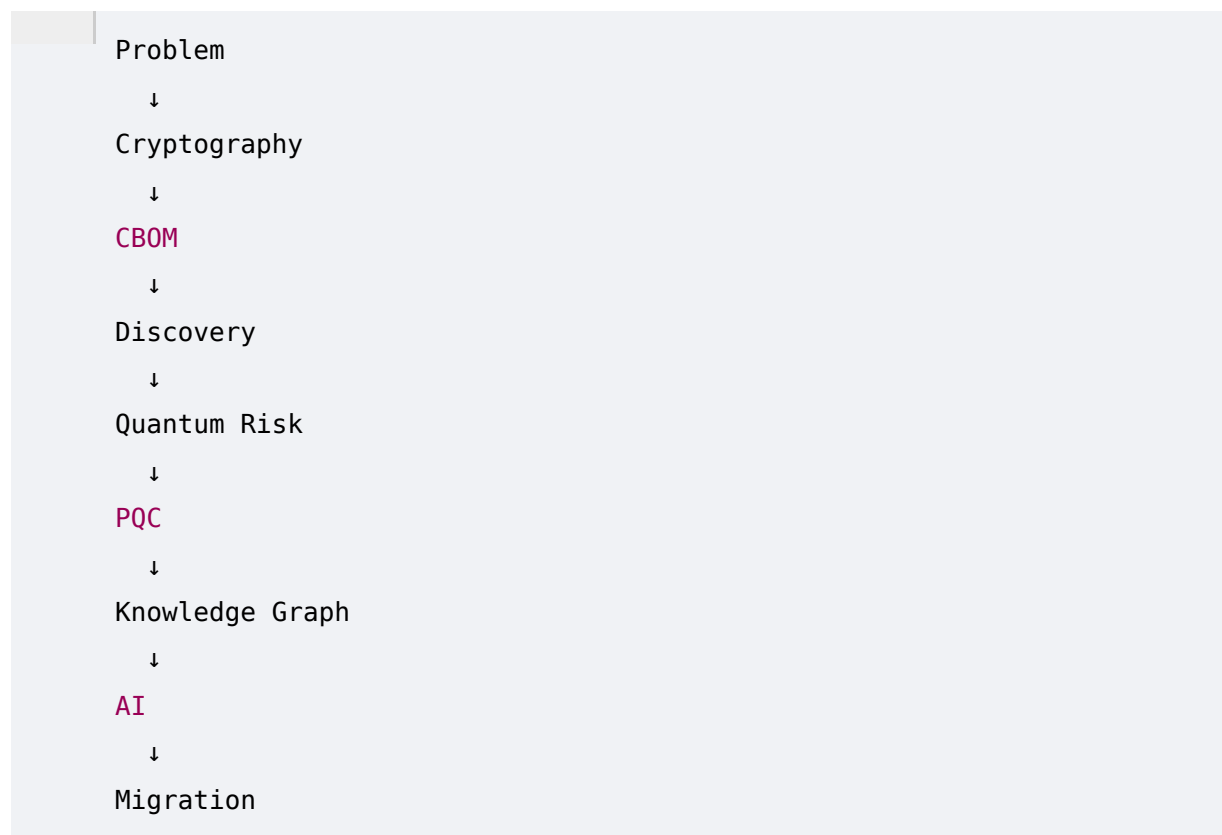
Theme: Blockchain & Cybersecurity

Phase: 22 of 22

Status: Final Strategic Blueprint

1. Purpose of Phase 22

We have now gone through the problem from almost every technical direction:



This final phase answers the most important practical question:

If our team actually chooses SIH26164, what exactly should we build, how should we build it, how should we demonstrate it, and how should we position it against competing teams?

The goal is not to build every feature imagined in the previous phases.

The goal is to build a **small but extremely deep vertical slice** that convincingly demonstrates the complete ECDAT vision.

2. The Final Product Definition

ECDAT should be positioned as:

An enterprise cryptographic discovery and quantum-readiness platform that automatically inventories cryptographic assets, maps their dependencies and business impact, evaluates quantum risk, recommends migration strategies, and verifies migration progress.

The core workflow:



That is the product.

3. The One Sentence Pitch

If a judge asks:

"What exactly did you build?"

Answer:

"ECDAT discovers cryptography across enterprise software and infrastructure, converts it into a cryptographic bill of materials, maps its business dependencies, calculates quantum migration risk, recommends suitable PQC or hybrid alternatives, and verifies remediation through rescanning."

Short.

Technical.

Clear.

4. The Problem in One Sentence

The fundamental problem is:

Organizations cannot migrate to post-quantum cryptography efficiently if they do not know where cryptography exists, how it is being used, what depends on it, and which assets need to be migrated first.

5. Why This Problem Is Hard

The enterprise may have:

- Source Code
- Libraries
- Containers
- Binaries
- Certificates
- TLS
- SSH
- HSMs
- Cloud Services
- APIs
- JWT
- Databases
- Applications
- Legacy Systems
- Third-Party Software

Cryptography is distributed across all of them.

There is no single:

```
crypto_inventory.json
```

waiting to be discovered.

6. The Core Insight

The most important insight behind ECDAT is:

Cryptographic migration is an inventory and dependency problem before it is an algorithm-selection problem.

You cannot simply say:

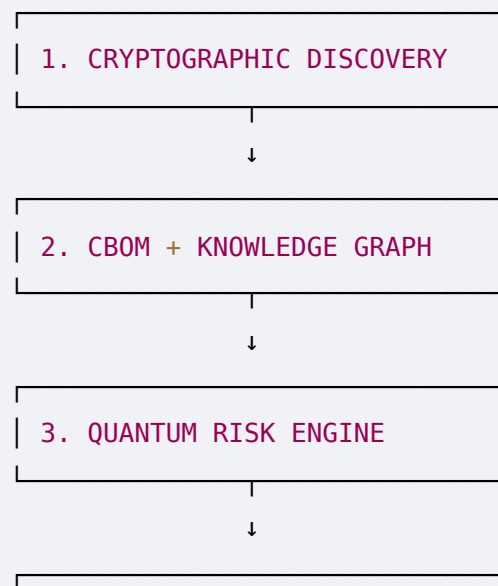
```
RSA → PQC
```

because you first need to know:

```
Where is RSA?
Why is it there?
Who depends on it?
What data does it protect?
What library implements it?
What hardware supports it?
What applications depend on it?
What breaks if it changes?
```

7. ECDAT's Five Core Engines

The final product should revolve around five engines:



4. PQC RECOMMENDATION ENGINE



AI sits across these engines as an analyst interface.

8. What You Should Actually Build

For the SIH prototype, prioritize:

MUST HAVE

- ✓ Source scanning
- ✓ Dependency scanning
- ✓ Certificate scanning
- ✓ Crypto detection
- ✓ **CBOM**
- ✓ Risk scoring
- ✓ Knowledge graph
- ✓ Dashboard
- ✓ **AI** analyst
- ✓ **PQC** recommendation
- ✓ Migration plan
- ✓ Rescan verification

SHOULD HAVE

- ✓ Container scanning
- ✓ Binary metadata analysis
- ✓ Policy engine
- ✓ Report generation
- ✓ Audit trail

NICE TO HAVE

- Advanced dynamic analysis
- **HSM** integration
- Cloud-native discovery
- Runtime instrumentation
- Air-gapped **AI**
- Advanced **ML** classification

Do not let the nice-to-have features destroy the MVP.

9. The Golden Rule

If you have limited time:

Build depth, not breadth.

One excellent scanner covering four languages is better than ten shallow scanners that produce unreliable findings.

10. Recommended SIH MVP

Build the following:

```
graph TD; A[Repository Upload] --> B[Source Scanner]; B --> C[Crypto Detection]; C --> D[CBOM]; D --> E[Risk Engine]; E --> F[Knowledge Graph]; F --> G[AI Analyst]; G --> H[PQC Recommendation]; H --> I[Migration Plan];
```

Repository Upload
↓
Source Scanner
↓
Crypto Detection
↓
CBOM
↓
Risk Engine
↓
Knowledge Graph
↓
AI Analyst
↓
PQC Recommendation
↓
Migration Plan

If this loop works end-to-end, you have a compelling prototype.

11. The Demo Application

Create a fictional enterprise:

"Astra Financial Technologies"

Do not rely exclusively on random GitHub repositories during the final demo.

Build a controlled dataset.

12. Astra Enterprise

Example:

Astra Financial Technologies
Applications: 1,000+
Repositories: 400+
Cryptographic Assets: 10,000+
Critical Applications: 150+

The numbers should be generated from the actual dataset.

Never fake performance metrics.

13. Business Units

Create:

Payments
Identity
Banking
Analytics
Infrastructure
Customer Platform
Internal Operations

14. Applications

Example:

Payment Gateway
Identity Service
Customer Portal
Transaction Processor

Secure File Transfer

API Gateway

Mobile Backend

Data Warehouse

15. Inject Cryptographic Problems

Create realistic technical debt:

RSA-2048

ECDSA

SHA-1

TLS 1.2

Legacy OpenSSL

Old certificates

Hardcoded algorithms

Legacy crypto provider

16. Create PQC-Ready Applications

Some applications should already use:

Modern cryptography

Hybrid approaches

Approved configurations

This gives the dashboard a maturity spectrum.

17. Final Dashboard

The first screen should immediately answer:

How much cryptography exists?

How much is quantum vulnerable?

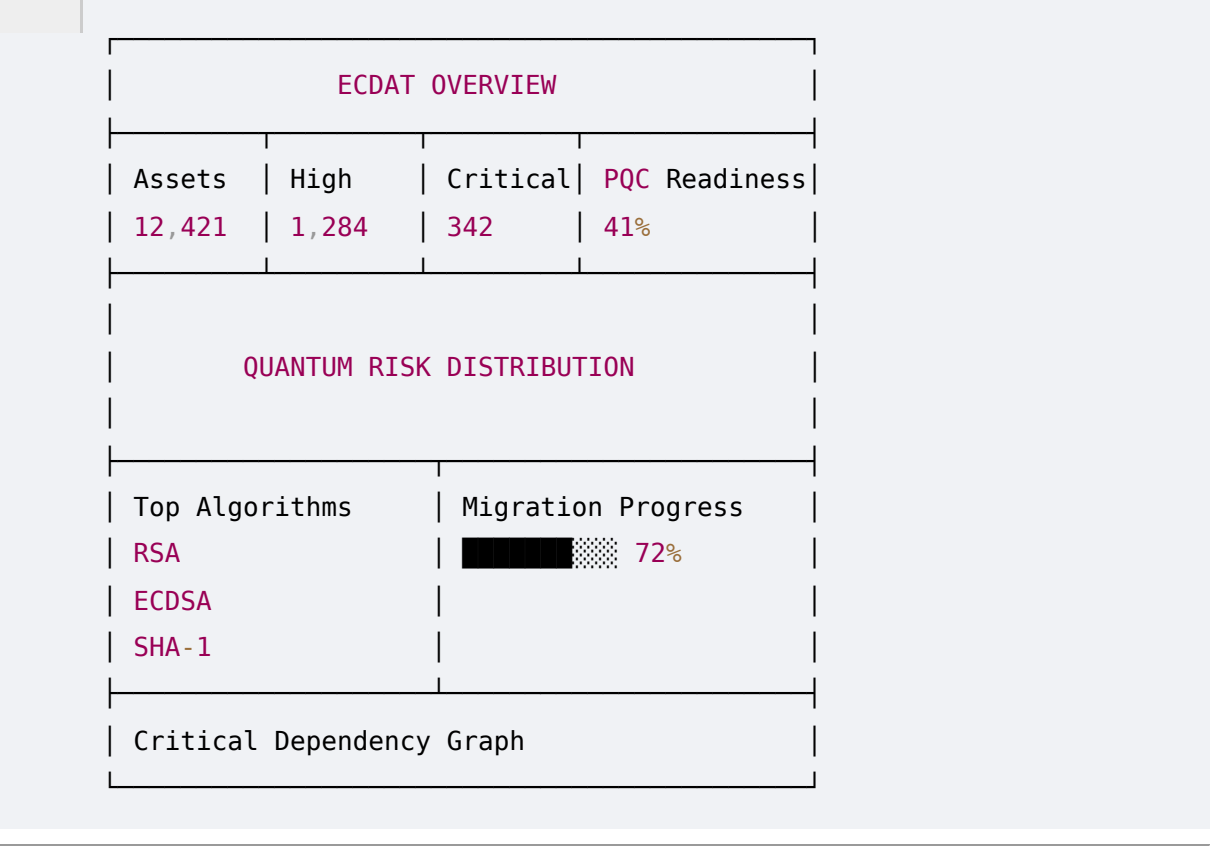
What is critical?

What should we migrate first?

How far along are we?

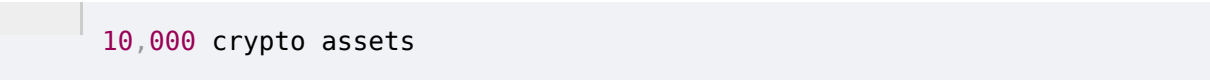
18. Dashboard Layout

Recommended:



19. The Most Important Dashboard Number

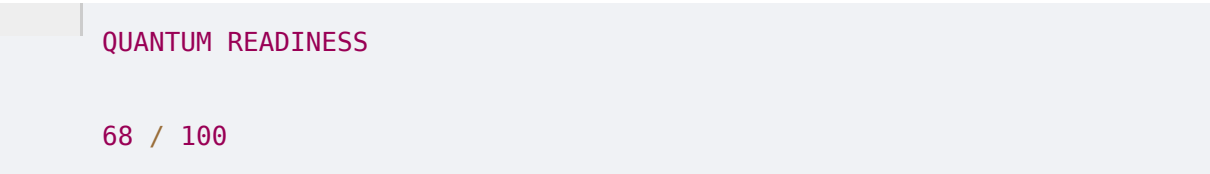
Do not only display:



Show:

Quantum Readiness Score

Example:



Then explain exactly how it was calculated.

20. Score Breakdown

Algorithm posture	25%
Inventory completeness	20%
Migration readiness	20%
Dependency readiness	15%
Certificate posture	10%
Policy compliance	10%

These weights are an example. Your final implementation should document and justify its chosen model.

21. Risk Heatmap

Show:

BUSINESS CRITICALITY	
	Low Medium High Critical
QUANTUM RISK	Low Medium High Critical

The top-right corner becomes the priority zone.

22. Top Risk List

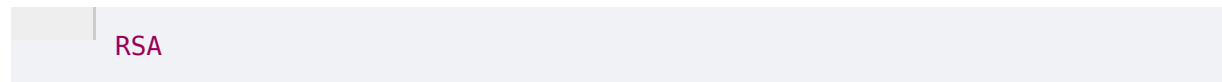
Example:

```
#1 Payment Gateway
RSA-2048
Critical data
Long retention
Migration dependency: HSM
Risk: Critical

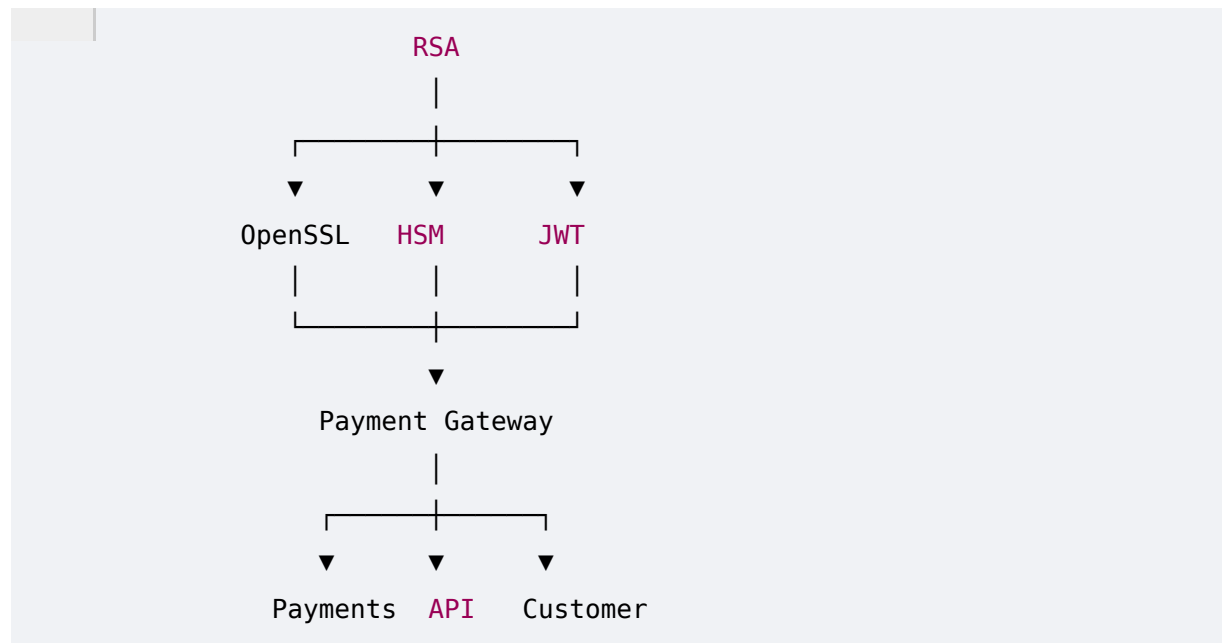
#2 Identity Provider
ECDSA
Long-lived credentials
Risk: High
```

23. The Killer Feature: Dependency Graph

Click:



Show:



24. Why the Graph Matters

A flat list tells you:

RSA exists.

The graph tells you:

Changing RSA may affect these 47 applications, 3 libraries, 2 HSMs and 12 business processes.

That is much more useful.

25. AI Analyst

The AI interface should not look like a generic ChatGPT clone.

It should look like:

ECDAT Security Analyst

26. Example AI Question

User:

"What are my five highest quantum risks?"

AI:

1. Payment Gateway
RSA-2048
Critical data
Long retention
2. Identity Service
ECDSA
Critical authentication dependency
3. Secure File Transfer
RSA
Long-lived encrypted archives
- ...

Every answer should reference actual ECDAT evidence.

27. Second AI Question

"Why is the Payment Gateway critical?"

Answer:

Because:

- RSA-2048 is quantum-vulnerable
- The application processes sensitive payment data
- Data may require long-term confidentiality
- Migration depends on a shared cryptographic provider
- 17 downstream services depend on this application

28. Third AI Question

"What should we migrate first?"

AI should rank based on:

Quantum vulnerability
+
Data lifetime

- +
Business criticality
- +
Exposure
- +
Migration complexity
- +
Dependency blast radius

29. Fourth AI Question

"What would happen if we replace OpenSSL version X?"

The graph should answer:

```
Applications affected:
47

Certificates affected:
18

Algorithms affected:
3

Business processes:
12

Migration blockers:
4
```

This is where ECDAT becomes genuinely interesting.

30. PQC Recommendation

Do not make the AI invent cryptographic algorithms.

Create an:

Approved Cryptographic Alternatives Registry.

31. Registry

Example:

- Algorithm
- Family
- Security Status
- PQC Status
- Supported Operations
- Compatibility
- Performance
- Migration Notes

32. Recommendation Engine

Use deterministic logic:

```
graph TD; A[Current algorithm] --> B[Usage]; B --> C[Security status]; C --> D[Target options]; D --> E[Compatibility]; E --> F[Policy]; F --> G[Migration cost]; G --> H[Recommendation];
```

AI then explains the result.

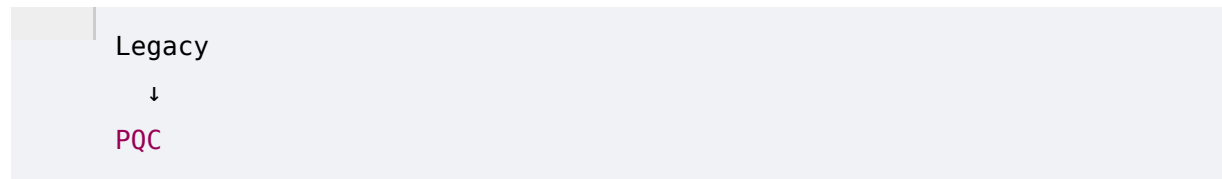
33. Hybrid Strategy

The product should recognize that migration may be staged.

Conceptually:

```
graph TD; A[Legacy] --> B[Hybrid]; B --> C[PQC];
```

rather than:



overnight.

34. Migration Planner

Example:

PAYMENT GATEWAY

Current:
RSA-2048

Target:
Approved hybrid strategy

Steps:

1. Upgrade provider
2. Validate compatibility
3. Introduce hybrid mechanism
4. Update certificate infrastructure
5. Test downstream services
6. Deploy to staging
7. Monitor
8. Production rollout
9. Post-mortem

35. Migration Dependencies

Show:

Migration cannot proceed until:

- ✓ Provider upgraded
- ✓ HSM supports target
- ✓ Certificate chain updated
- ✓ Client compatibility confirmed

36. Migration Ownership

Every task should have:

Owner
Priority
Status
Dependency
Deadline

37. Verification

After migration:

RESCAN

ECDAT compares:

Before
vs
After

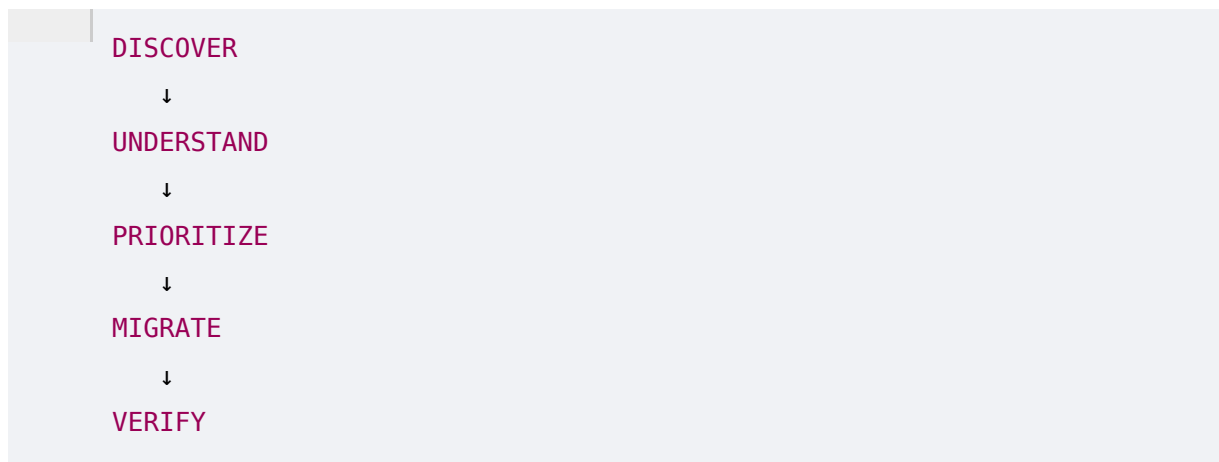
38. Verification Dashboard

Example:

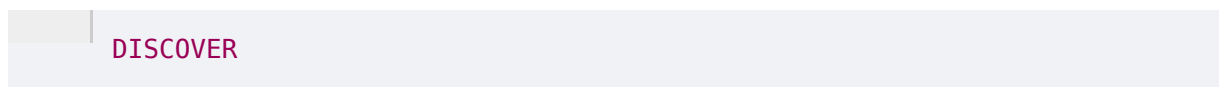
Payment Gateway
Before:
RSA-2048
Risk: Critical
After:
Approved target
Risk: Medium
Migration:
VERIFIED

39. This Creates a Closed Loop

This is the final product story:



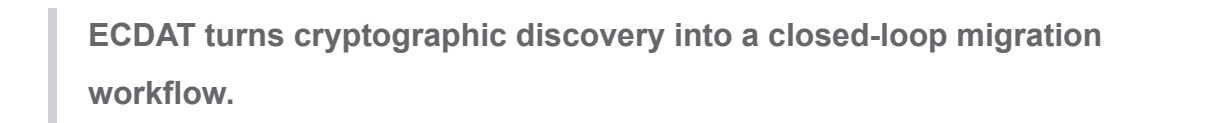
Most inventory tools stop at:



ECDAT should continue beyond that.

40. The "Closed-Loop Security" Position

Use:



This is one of the strongest positioning statements.

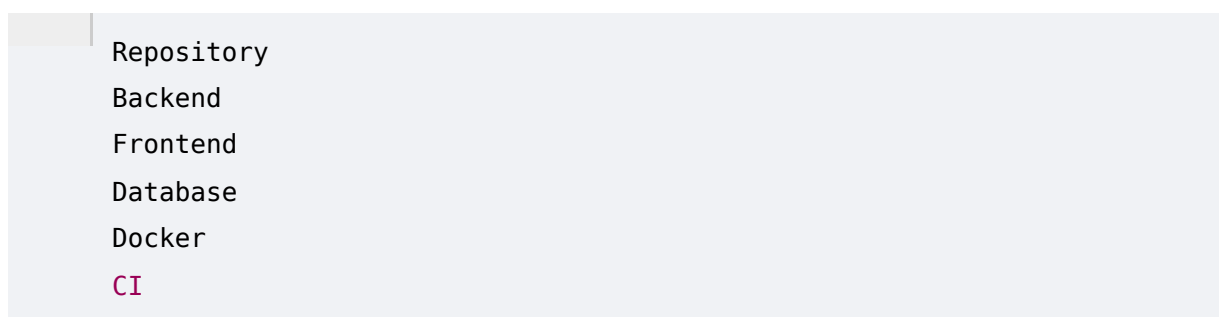
41. Execution Strategy

Now we move from product design to team execution.

Do not build everything simultaneously.

42. Stage 1 — Foundation

Build:



Deliverable:

System launches successfully.

43. Stage 2 — Scanner

Build:

Source scanner
Dependency scanner
Certificate parser

Deliverable:

Repository
↓
Findings

44. Stage 3 — CBOM

Build:

Normalizer
Crypto Asset model
Evidence model
CBOM API

Deliverable:

Findings
↓
CBOM

45. Stage 4 — Risk

Build:

Risk engine
Mosca-inspired prioritization
Business criticality
Data lifetime

Deliverable:

CBOM



Risk

46. Stage 5 — Graph

Build:

Neo4j

Graph ingestion

React Flow visualization

Deliverable:

Asset



Dependencies



Business impact

47. Stage 6 — AI

Build:

Retrieval

Context generation

AI Gateway

Structured responses

Deliverable:

Ask ECDAT



Evidence-backed answer

48. Stage 7 — Recommendation

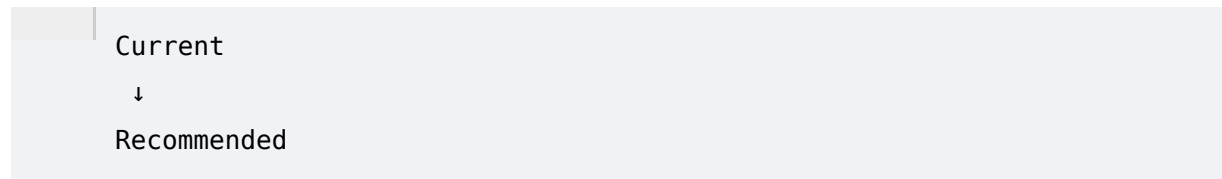
Build:

Algorithm registry

Policy engine

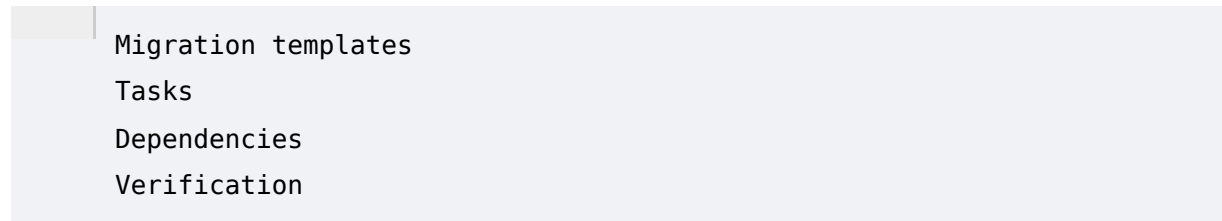
Recommendation engine

Deliverable:

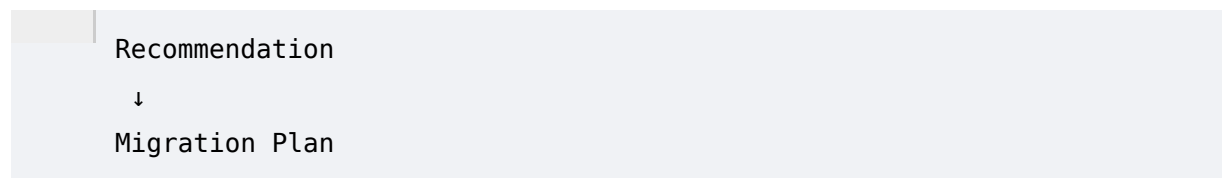


49. Stage 8 — Migration

Build:

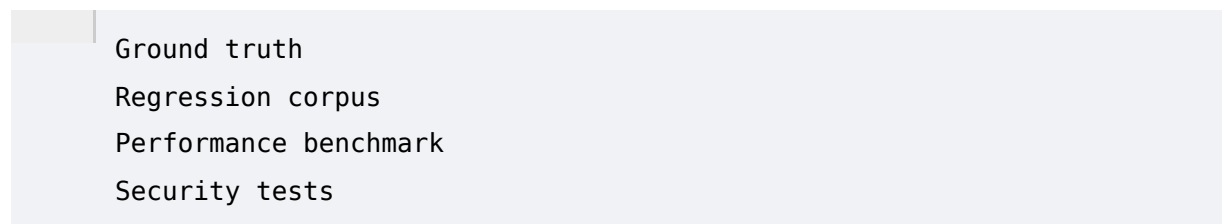


Deliverable:



50. Stage 9 — Testing

Build:

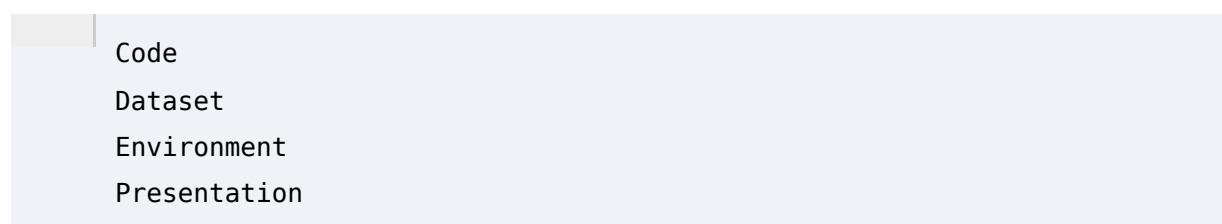


Deliverable:



51. Stage 10 — Demo

Freeze:



Then stop adding major features.

52. Feature Freeze

This is extremely important.

Before SIH:

Do not keep adding features until the final day.

Instead:

Feature Freeze

↓

Bug Fix

↓

Performance

↓

Demo Stability

53. Six-Person Team Structure

For a six-person team:

Person 1 — Tech Lead / Backend

Own:

Architecture

FastAPI

Database

API

Integration

54. Person 2 — Discovery Engineer

Own:

Source scanner

AST

Rules

Dependency detection

55. Person 3 — Security / CBOM Engineer

Own:

- CBOM
- Risk
- Policies
- Certificate analysis

56. Person 4 — Graph / Data Engineer

Own:

- Neo4j
- Graph model
- Blast radius
- Business mapping

57. Person 5 — Frontend Engineer

Own:

- Dashboard
- CBOM UI
- Graph UI
- Migration UI

58. Person 6 — AI / Intelligence Engineer

Own:

- AI Gateway
- RAG
- Recommendation
- Migration generation
- AI security

59. Shared Responsibility

Everyone should understand:

```
Problem Statement
Architecture
Demo
Security
Limitations
```

A judge can ask anyone anything.

60. Team Documentation

Maintain:

```
README.md
ARCHITECTURE.md
CBOM.md
SCANNER.md
RISK_MODEL.md
AI.md
SECURITY.md
TESTING.md
DEMO.md
```

61. One Source of Truth

Do not allow:

```
Developer A:
RSA risk = 80

Developer B:
RSA risk = 90
```

Document the actual scoring model.

62. Definition of Done

A feature is not done when:

```
"It works on my laptop."
```

It is done when:

- ✓ Code
- ✓ Tests
- ✓ Documentation
- ✓ Error handling
- ✓ Security
- ✓ Demo integration

63. SIH Presentation Structure

A strong presentation:

1. Problem
2. Why existing approaches fail
3. ECDAT
4. Architecture
5. Live Demo
6. Innovation
7. Validation
8. Security
9. Impact
10. Roadmap

64. Slide 1 — Problem

Large statement:

"You cannot migrate what you cannot see."

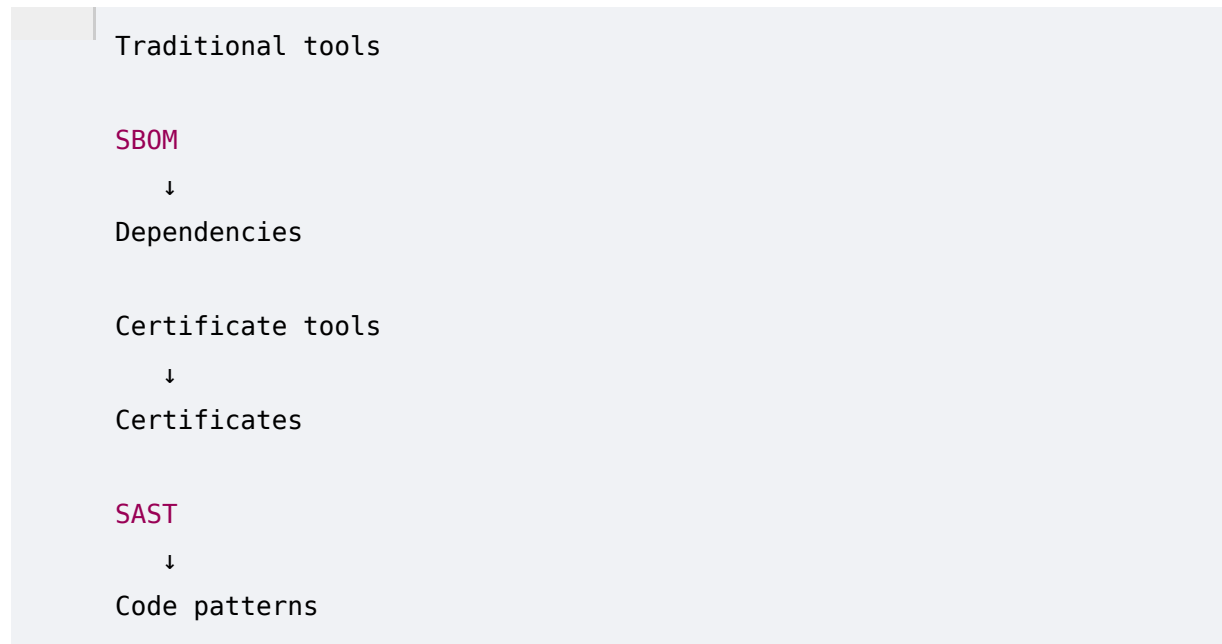
Then:

Enterprise cryptography is fragmented across:

Source
Libraries
Binaries
Containers
Certificates
Infrastructure
Cloud

65. Slide 2 — The Gap

Show:



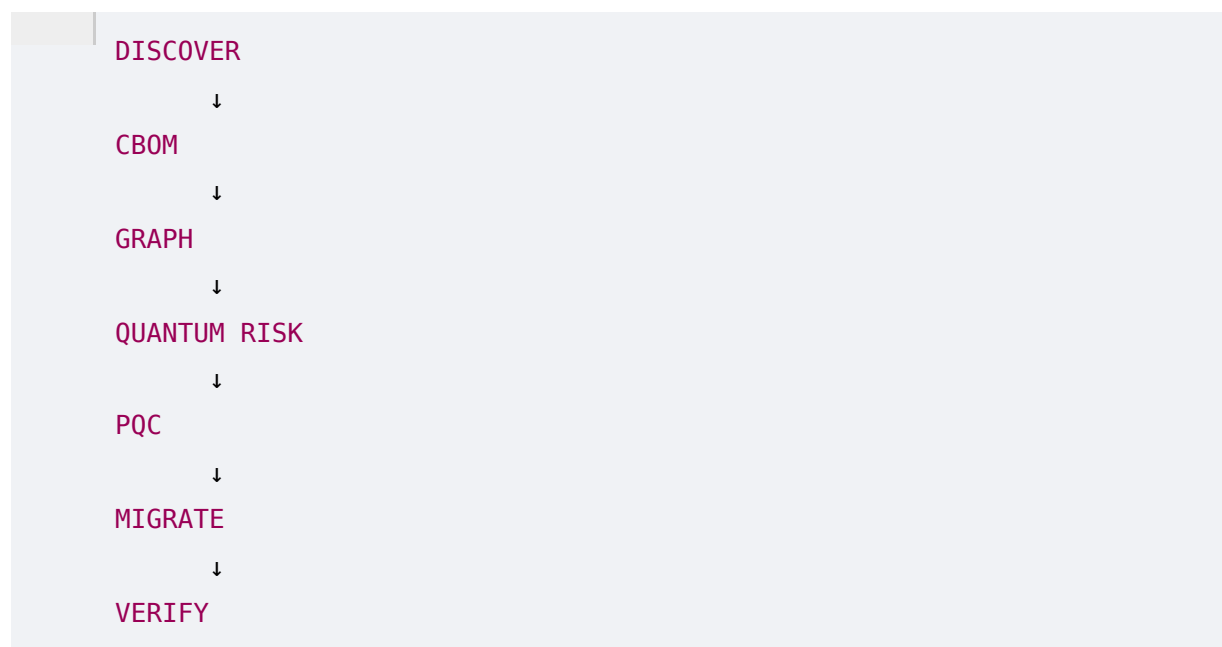
Then:

Nobody gives the complete cryptographic migration picture.

Be careful with absolute claims. Phrase it as the gap your solution addresses.

66. Slide 3 — ECDAT

Show:



67. Slide 4 — Architecture

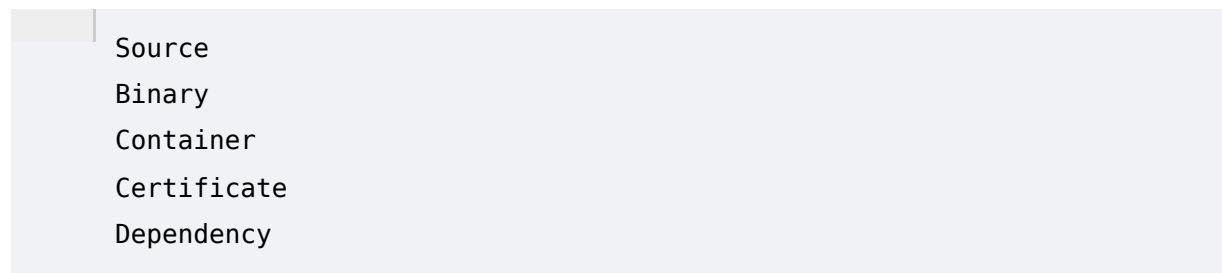
Use the architecture diagram.

Do not put 50 boxes.

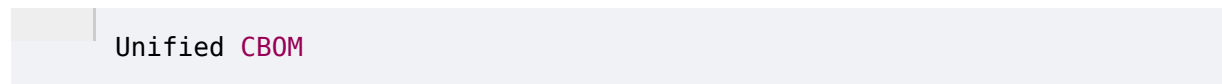
The judge needs to understand the flow in 10 seconds.

68. Slide 5 — Discovery

Show:

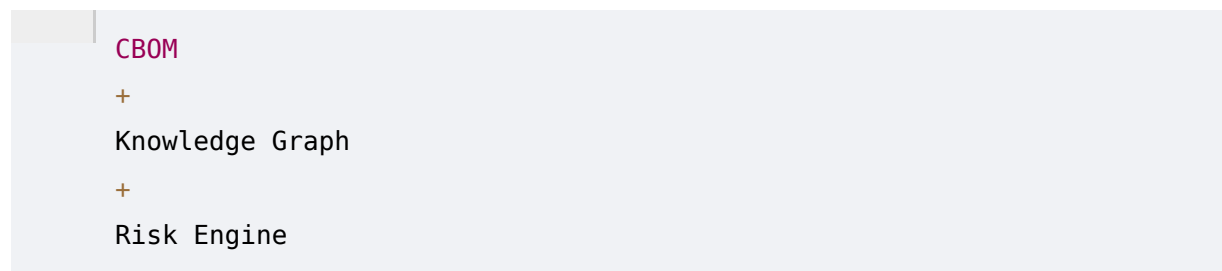


feeding:

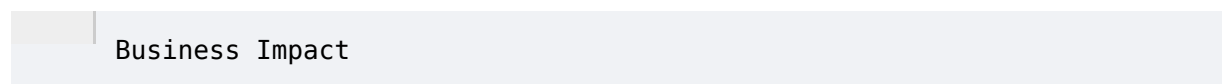


69. Slide 6 — Intelligence

Show:



→



70. Slide 7 — AI

Don't say:

"We use AI."

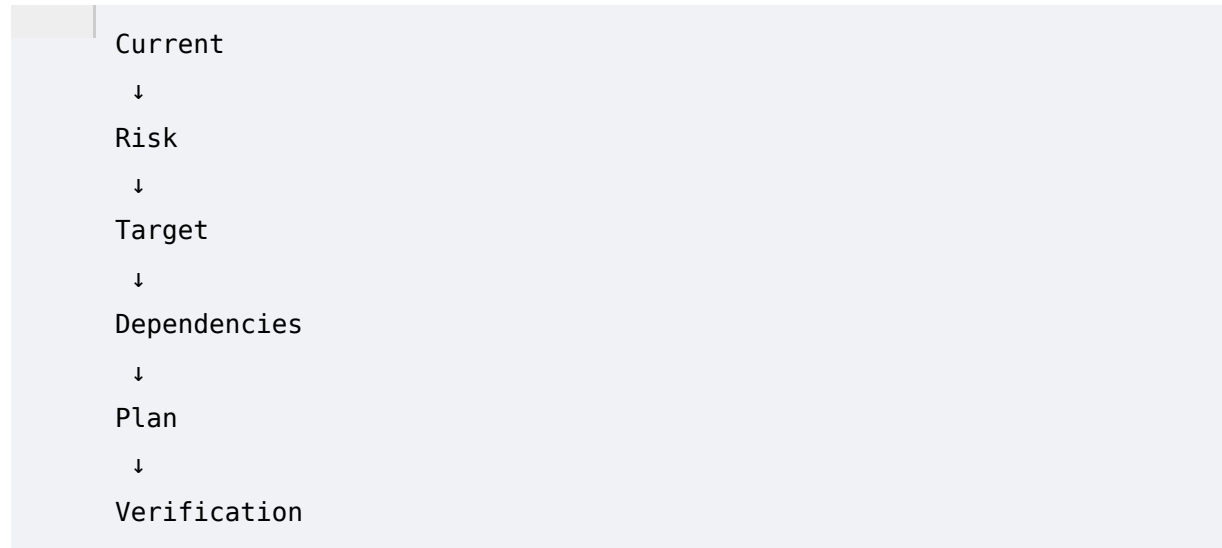
Say:

"AI is grounded in ECDAT's CBOM, graph, evidence and policy layer."

This distinction matters.

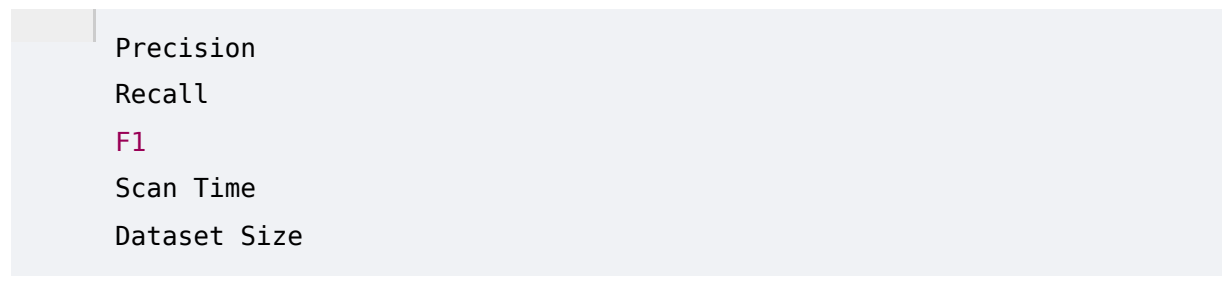
71. Slide 8 — Migration

Show:



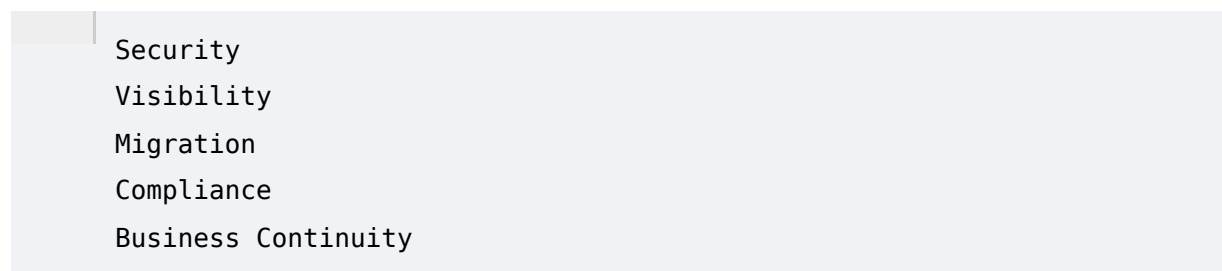
72. Slide 9 — Validation

Show actual:



73. Slide 10 — Impact

Show:



74. The Five-Minute Demo

A five-minute demo could be:

0:00–0:30

Problem.

0:30–1:00

Launch ECDAT.

1:00–1:45

Run/show discovery.

1:45–2:30

CBOM.

2:30–3:15

Graph + blast radius.

3:15–4:00

Risk + AI.

4:00–4:40

Migration plan.

4:40–5:00

Rescan + verification.

75. Don't Waste Demo Time

Avoid:

- Login
- Menu navigation
- Loading screens
- Configuration
- Long scans

Prepare the environment beforehand.

76. The 30-Second Hook

Start with:

"We have built a system that answers a question most organizations cannot answer today: **where exactly is cryptography being used across the**

enterprise, and what would happen if we had to migrate it?"

Then immediately show the graph.

77. Judge Question: "Why not just use an SBOM?"

Answer:

"An SBOM tells you what software components exist. ECDAT focuses on what cryptographic mechanisms exist inside those components, how they're used, what data and applications depend on them, and what migration risk they create."

78. Judge Question: "Why CBOM?"

Answer:

"Because software-component inventory and cryptographic inventory answer different questions. A dependency can exist without revealing which cryptographic algorithms, certificates, protocols or key configurations are actually relevant to migration."

79. Judge Question: "Why AI?"

Answer:

"The deterministic engines establish the facts. AI sits on top of those facts to let analysts ask natural-language questions, understand evidence, prioritize findings and generate migration guidance."

80. Judge Question: "What prevents hallucinations?"

Answer:

"The AI does not determine the underlying inventory. Its context is retrieved from ECDAT's CBOM, graph, evidence and policy layers, and high-impact decisions remain human-approved."

81. Judge Question: "What if AI goes down?"

Answer:

"ECDAT's core discovery, CBOM, risk and graph functions are deterministic and operate independently of the AI layer."

Excellent answer.

82. Judge Question: "Can you scan binaries?"

Answer:

"Yes, through a dedicated binary analysis pipeline. The prototype emphasizes static analysis and metadata extraction rather than executing untrusted binaries."

83. Judge Question: "Can you scan containers?"

Answer:

"Yes. We inspect image manifests, layers, packages, binaries and cryptographic configuration without requiring arbitrary execution of the image."

84. Judge Question: "Can you scan hardware?"

Answer:

Do not overclaim.

Say:

"Our architecture supports hardware and HSM metadata as inventory sources, while direct vendor-specific hardware introspection is an extensibility layer rather than something we claim to universally support in the prototype."

Honesty builds credibility.

85. Judge Question: "What about proprietary systems?"

Answer:

"The scanner framework is plugin-based. Where source or runtime access is unavailable, ECDAT can ingest supported metadata, SBOMs, certificates, configuration exports and vendor APIs."

86. Judge Question: "How do you calculate quantum risk?"

Answer:

"We combine cryptographic vulnerability, data lifetime, migration time, business criticality and exposure, with a Mosca-inspired prioritization model. The weights and thresholds are configurable and versioned."

87. Judge Question: "Why Mosca?"

Answer:

"Because the urgency of migration is not determined only by whether an algorithm is quantum-vulnerable. It depends on how long the protected information must remain confidential and how long migration will take."

88. Judge Question: "What is the actual innovation?"

Strong answer:

"The innovation is the integration layer: cryptographic discovery becomes a connected enterprise graph, that graph feeds quantum-risk prioritization, and the resulting intelligence drives evidence-backed migration planning and verification."

89. Judge Question: "Why blockchain theme?"

This is important because the problem is under:

Blockchain & Cybersecurity.

Answer:

"The problem is fundamentally cybersecurity and cryptographic infrastructure. While blockchain is not required to implement ECDAT, the platform's cryptographic inventory and migration intelligence directly address the cryptographic foundations shared across blockchain, PKI, authentication and enterprise security systems."

Do not force blockchain into the architecture unnecessarily.

90. Judge Question: "Why not build a blockchain?"

Answer:

"Because blockchain is not the core requirement of the problem statement. Adding blockchain purely for thematic alignment would increase complexity without improving cryptographic discovery or migration."

That is a very strong engineering answer.

91. Judge Question: "How accurate is it?"

Answer with actual numbers.

Never improvise.

Use:

Our ground-truth corpus:

X artifacts

Precision:

Y%

Recall:

Z%

F1:

W%

92. Judge Question: "What are your limitations?"

Say:

"Dynamic runtime cryptography hidden inside proprietary systems and hardware-specific implementations are harder to discover without runtime or vendor integration. We therefore expose confidence and evidence rather than pretending every finding is certain."

That answer can impress judges.

93. Judge Question: "Is this production ready?"

Do not say:

"Yes."

Instead:

"The prototype demonstrates the core workflow. Production deployment would add enterprise SSO, HSM/KMS integration, SIEM integration, high availability, deeper runtime discovery and additional vendor connectors."

94. Judge Question: "How scalable is it?"

Answer architecturally:

```
API
↓
Queue
↓
Horizontal scanner workers
↓
PostgreSQL
+
Graph
+
Object storage
```

Then cite measured benchmarks.

95. Judge Question: "How do you secure source code?"

Answer:

```
Encrypted storage
RBAC
Least privilege
Sandboxed scanning
Evidence minimization
Secret redaction
Audit logging
Private deployment
```

96. Judge Question: "Can it run offline?"

Answer:

"The architecture supports private and air-gapped deployment profiles. The AI layer can be replaced with a local or enterprise-hosted model, while the deterministic discovery and risk engines do not require internet connectivity."

97. Judge Question: "How do you avoid exposing private keys?"

Answer:

"ECDAT is designed primarily to discover cryptographic metadata rather than collect private key material. Private-key acquisition is avoided unless an explicit enterprise integration requires it."

98. Judge Question: "What happens after discovery?"

This is one of your strongest answers:

"Discovery is only phase one. ECDAT connects the inventory to risk, business impact, PQC recommendations, migration tasks and post-migration verification."

99. Judge Question: "Why do we need the graph?"

Answer:

"Because cryptographic assets are not independent. A single provider, certificate authority, library or algorithm can affect hundreds of applications. The graph reveals that blast radius."

100. Judge Question: "What is your biggest technical challenge?"

Strong answer:

"Cryptography is rarely explicit in one place. The hardest part is correlating weak signals across source code, dependencies, binaries, certificates and infrastructure into one evidence-backed asset model."

101. Judge Question: "How does AI know what to recommend?"

Answer:

"The recommendation engine first filters candidate alternatives through deterministic policy, compatibility and risk rules. AI then explains and contextualizes those recommendations rather than inventing cryptographic policy."

102. Judge Question: "Can AI automatically migrate code?"

Answer:

"Not by default. We generate migration guidance and verification steps, while production changes remain human-approved."

103. Judge Question: "Can it detect a cryptographic algorithm hidden behind a wrapper?"

Answer:

"Our AST and dependency analysis attempts to resolve aliases and common wrappers. We expose confidence and evidence when the relationship is uncertain."

104. Judge Question: "How do you deal with false positives?"

Answer:

"Every finding carries evidence and confidence. Analysts can mark false positives, and those cases become regression tests and rule

improvements."

105. Judge Question: "How do you deal with false negatives?"

Answer:

"We maintain a ground-truth corpus, track recall by algorithm and language, and continuously expand adversarial and real-world test cases."

106. The Strongest Technical Phrase

Use:

Evidence-backed cryptographic intelligence.

It summarizes the philosophy.

107. The Strongest Product Phrase

Use:

Discover. Assess. Migrate. Verify.

108. The Strongest Problem Phrase

Use:

You cannot migrate what you cannot see.

109. The Strongest AI Phrase

Use:

AI explains the evidence; it does not invent the evidence.

110. The Strongest Security Phrase

Use:

ECDAT treats itself as security-critical infrastructure.

111. The Strongest Architecture Phrase

Use:

Deterministic security core, AI-assisted analyst layer.

112. The Strongest Differentiation

Use:

"We are not building another vulnerability scanner. We are building a cryptographic migration intelligence layer."

113. What NOT to Say

Avoid:

- ✗ "Our AI detects everything."
- ✗ "100% accurate."
- ✗ "No existing tool can do this."
- ✗ "Fully automated migration."
- ✗ "Quantum computers will break everything tomorrow."
- ✗ "Blockchain makes it secure."

These invite difficult questions.

114. What to Say Instead

Use:

- ✓ Evidence-backed
 - ✓ Configurable
 - ✓ Measurable
 - ✓ Explainable
 - ✓ Extensible
 - ✓ Enterprise-deployable
 - ✓ Human-in-the-loop
-

115. SIH Scoring Strategy

Regardless of the exact judging rubric, optimize around:

- Problem Understanding
- Technical Innovation
- Working Prototype
- Feasibility
- Scalability
- Security
- User Impact
- Presentation

116. Problem Understanding

Prove that you understand:

- Crypto inventory
- +
- Migration complexity
- +
- Data lifetime
- +
- Business impact

not just:

- PQC algorithms.

117. Innovation

Your innovation is the pipeline:

- Crypto Discovery
- +
- CBOM
- +
- Graph
- +
- Quantum Risk
- +
- PQC Recommendation

+
Migration
+
Verification

118. Working Prototype

Judges should be able to click:

Scan

and see:

Real findings

not a fake animation.

119. Feasibility

Show:

Modular scanners
API architecture
Plugin system
Private deployment

120. Scalability

Show:

Queue
Workers
Database
Graph
Object storage

121. Security

Show:

RBAC
Sandbox
Encryption

Audit
AI isolation

122. User Impact

Show:

Security team
Developers
Architects
Compliance
Executives

all get different value.

123. Executive Value

Executives get:

Risk
Impact
Cost
Progress

124. Security Team Value

Security teams get:

Inventory
Evidence
Risk
Dependencies

125. Developer Value

Developers get:

Exact file
Exact crypto API
Recommended alternative
Migration task

126. Architect Value

Architects get:

- Blast radius
- Dependencies
- Provider impact
- Migration sequencing

127. Compliance Value

Compliance gets:

- Policy violations
- Audit trail
- Evidence
- Reports

128. Long-Term Product

After SIH, ECDAT could evolve into:

- Crypto Discovery Platform
- +
- Quantum Readiness Platform
- +
- Crypto Governance Platform

129. Commercialization Potential

Potential customers:

- Banks
- Defense
- Government
- Healthcare
- Telecom
- Cloud providers
- Large enterprises
- Security consultancies

130. Enterprise Pricing Possibilities

Potential model:

Applications monitored
+
Crypto assets
+
Scanning frequency
+
Enterprise connectors

Do not focus heavily on pricing during SIH unless asked.

131. Future Connectors

ECDAT could eventually connect to:

GitHub
GitLab
Bitbucket
Jenkins
Kubernetes
AWS
Azure
GCP
Vault
HSMs
SIEM
CMDB
ServiceNow

132. Future Runtime Discovery

Eventually:

Static discovery
+
Runtime telemetry

could reveal actual cryptographic execution.

133. Future Enterprise Integration

ECDAT could integrate with:

```
graph TD; ECDAT --- CMDB; ECDAT --- AssetManagement[Asset Management]; ECDAT --- Ticketing; ECDAT --- IAM; ECDAT --- SIEM; ECDAT --- DevSecOps;
```

CMDB
Asset Management
Ticketing
IAM
SIEM
DevSecOps

134. Future Autonomous Capabilities

Long term:

```
graph TD; A[Discover] --> B[Recommend]; B --> C[Create ticket]; C --> D[Developer fixes]; D --> E[Rescan]; E --> F[Close ticket];
```

Discover
↓
Recommend
↓
Create ticket
↓
Developer fixes
↓
Rescan
↓
Close ticket

Still human-approved for high-risk actions.

135. Future PQC Readiness Simulation

A future version could simulate:

"What happens if we migrate this entire application portfolio?"

Graph simulation:

```
graph TD; A[Current Architecture] --> B[Migration Simulation]; B --> C[Compatibility Impact];
```

Current Architecture
↓
Migration Simulation
↓
Compatibility Impact
↓

Cost



Risk Reduction

136. Future Digital Twin

Ultimately ECDAT could become:

A digital twin of enterprise cryptographic infrastructure.

The organization could ask:

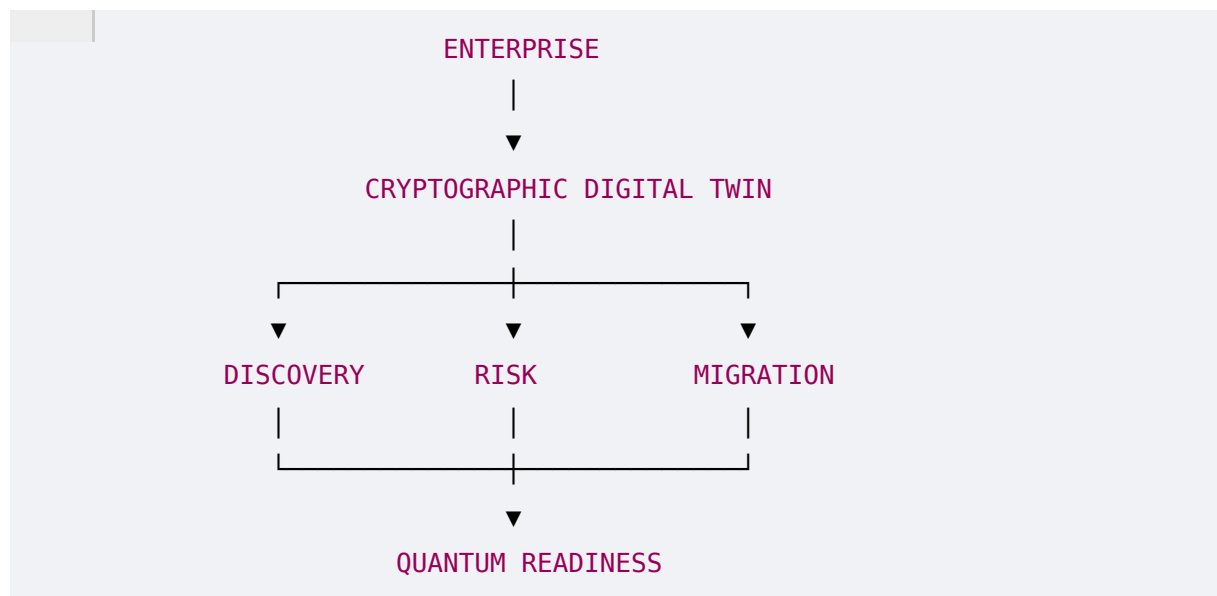
"What happens if this cryptographic provider becomes unsupported?"

or:

"What happens if we migrate this application to a PQC-hybrid scheme?"

137. Strategic Vision

Long-term:



138. What Makes ECDAT More Than a Scanner

A scanner answers:

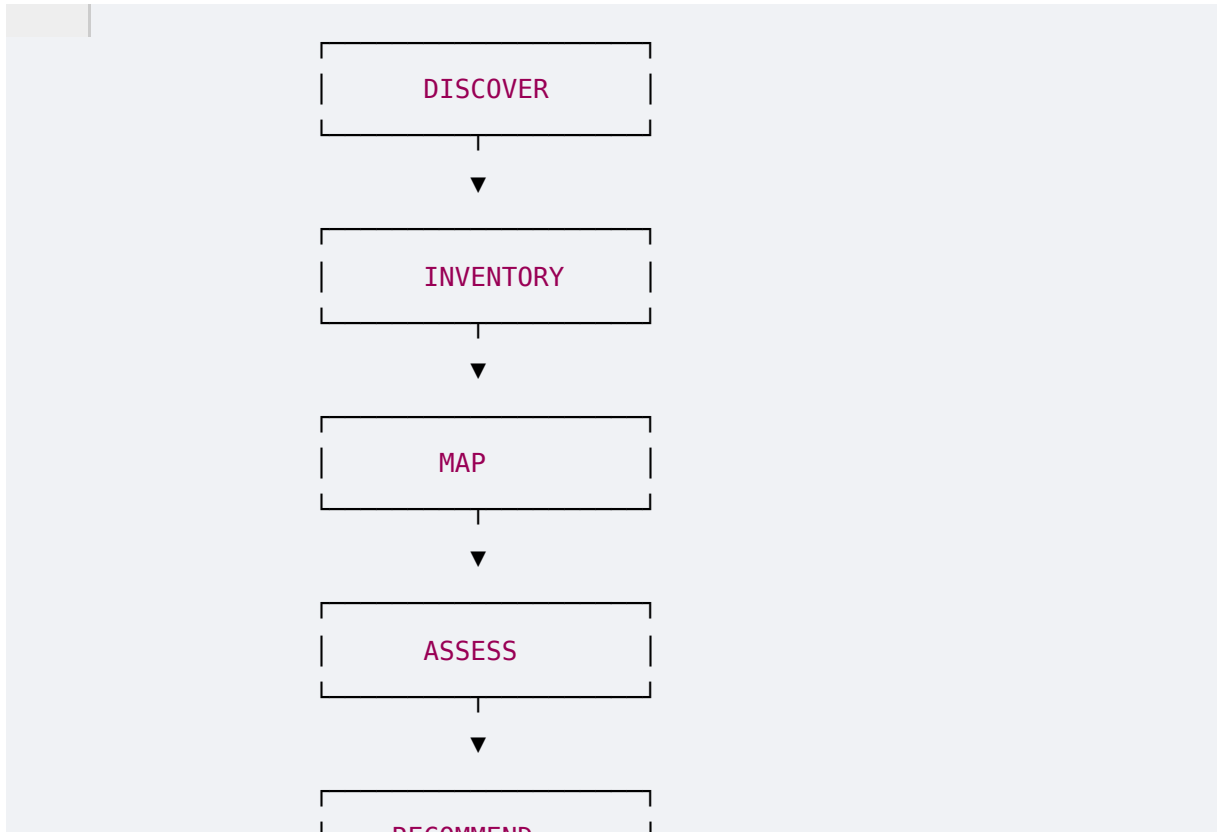
"What did I find?"

ECDAT should answer:

"What did I find, why does it matter, what depends on it, what should I do, and how do I know I've fixed it?"

That is the complete value proposition.

139. Final Product Loop



This creates continuous cryptographic governance.

140. FINAL TECHNICAL CHECKLIST

Before declaring the prototype complete:

Discovery

- Source scanner works
- Dependency scanner works
- Certificate parser works
- Container scanning works
- Binary pipeline exists
- Evidence captured

CBOM

- Assets normalized
- Versions captured
- Usage captured
- Confidence captured
- Evidence linked

Risk

- Algorithm risk
- Data lifetime
- Business criticality
- Migration effort
- Exposure
- Priority score

Graph

- Application dependencies
- Library relationships
- Algorithm relationships
- Business relationships
- Blast radius

AI

- RAG
- Evidence grounding
- Structured outputs
- Prompt injection defense
- Human approval

Migration

- Recommendation
- Compatibility
- Blockers
- Tasks
- Verification

Security

- Authentication
- Authorization
- Sandbox
- Encryption
- Audit
- Secrets management

Testing

- Ground truth
- Precision
- Recall
- F1
- Performance
- Security
- [Regression

141. FINAL DEMO CHECKLIST

The night before the presentation:

```
[ ] Database seeded
[ ] Graph seeded
[ ] Demo repository available
[ ] AI model working
[ ] Internet-independent fallback
[ ] Backup dataset
[ ] Backup screenshots
[ ] Backup video
[ ] Presentation exported
[ ] Architecture diagram ready
[ ] Metrics verified
[ ] All numbers accurate
```

142. DEMO BACKUP STRATEGY

Have three layers:

Level 1

Live application.

Level 2

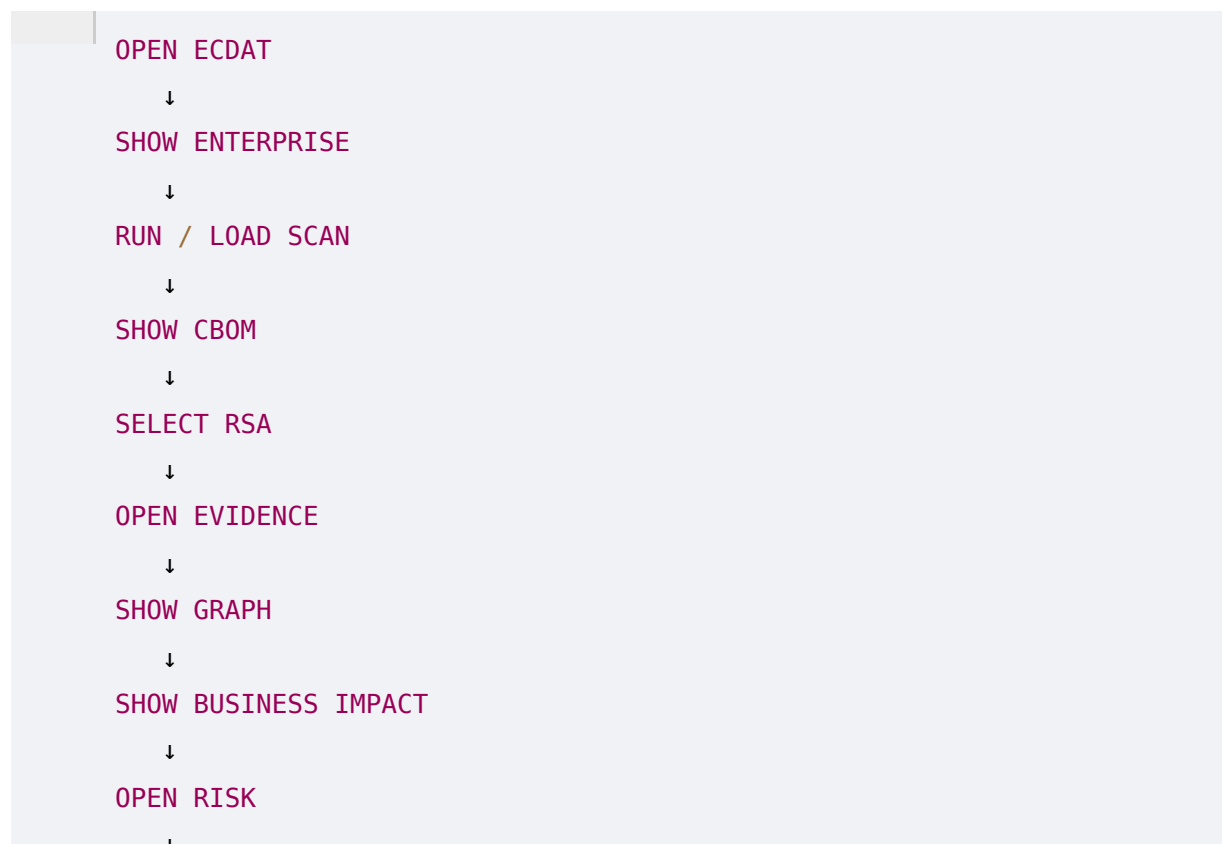
Precomputed dataset.

Level 3

Recorded demo.

Never depend on one fragile system.

143. The Final Demo Flow



144. Final Pitch — 60 Seconds

"Cryptographic migration has a visibility problem. Enterprises use cryptography across source code, libraries, containers, binaries, certificates and infrastructure, but rarely have a unified inventory of where those mechanisms exist or what depends on them.

ECDAT solves this by discovering cryptographic artifacts and converting them into a Cryptographic Bill of Materials. We then connect those assets through a knowledge graph to understand application and business

dependencies, evaluate quantum risk using factors such as data lifetime and migration effort, and identify which systems should be prioritized.

Our AI analyst is grounded in that evidence and helps security teams understand findings and evaluate migration options. ECDAT then generates PQC or hybrid migration plans and verifies the result through rescanning.

So we are not just finding cryptography. We are turning cryptographic discovery into a closed-loop migration workflow: Discover, Assess, Migrate and Verify."

145. Final Pitch — 2 Minutes

"The problem we are addressing is simple to state but extremely difficult to execute: organizations cannot migrate to post-quantum cryptography if they don't know where cryptography exists.

Cryptographic mechanisms are distributed across application source code, third-party libraries, binaries, containers, certificates, protocols, hardware and cloud infrastructure.

Existing security tools can provide pieces of this picture, but the migration problem requires those pieces to be connected.

ECDAT creates that connection.

First, our discovery engine scans enterprise artifacts and identifies algorithms, libraries, certificates, protocols and cryptographic usage.

Those findings are normalized into a Cryptographic Bill of Materials, with evidence and confidence.

Then we construct a knowledge graph connecting cryptographic assets to applications, dependencies, data and business processes.

Our quantum-risk engine combines cryptographic status with data lifetime, business criticality, exposure and migration effort to prioritize what should actually be addressed first.

On top of this deterministic security layer, our AI analyst allows security teams to ask natural-language questions and receive evidence-backed explanations.

Finally, ECDAT recommends suitable migration strategies, generates migration plans and rescans the environment to verify that remediation actually happened.

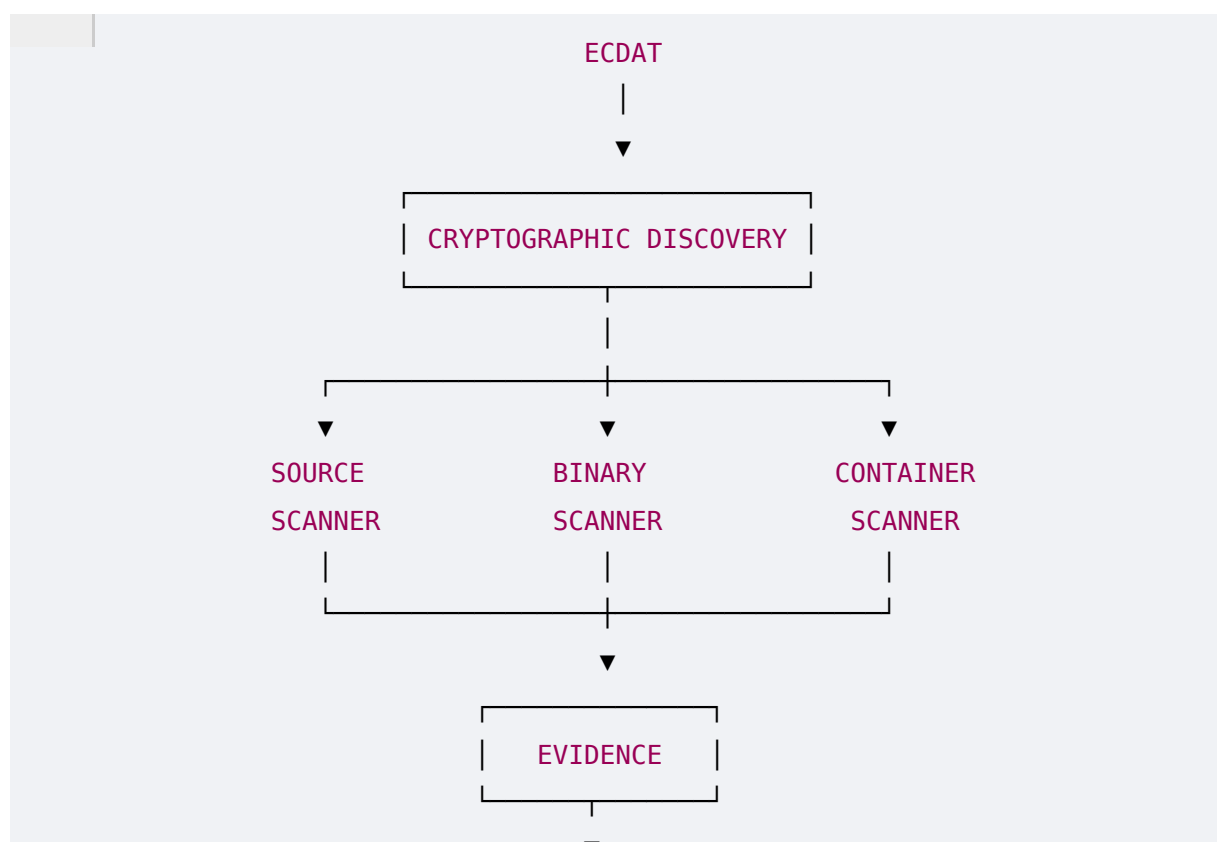
The result is not another dashboard and not simply an AI chatbot.

It is a cryptographic migration intelligence platform.

Discover. Assess. Migrate. Verify.

Because you cannot migrate what you cannot see."**

146. The Final Architecture in One Picture



147. FINAL ECDAT PRODUCT PHILOSOPHY

ECDAT should be built around five principles:

1. Evidence over assumptions

Every important finding should trace back to evidence.

2. Deterministic security over AI guesswork

The security core should not depend on an LLM.

3. Context over isolated findings

Knowing that RSA exists is less useful than knowing what depends on it.

4. Action over dashboards

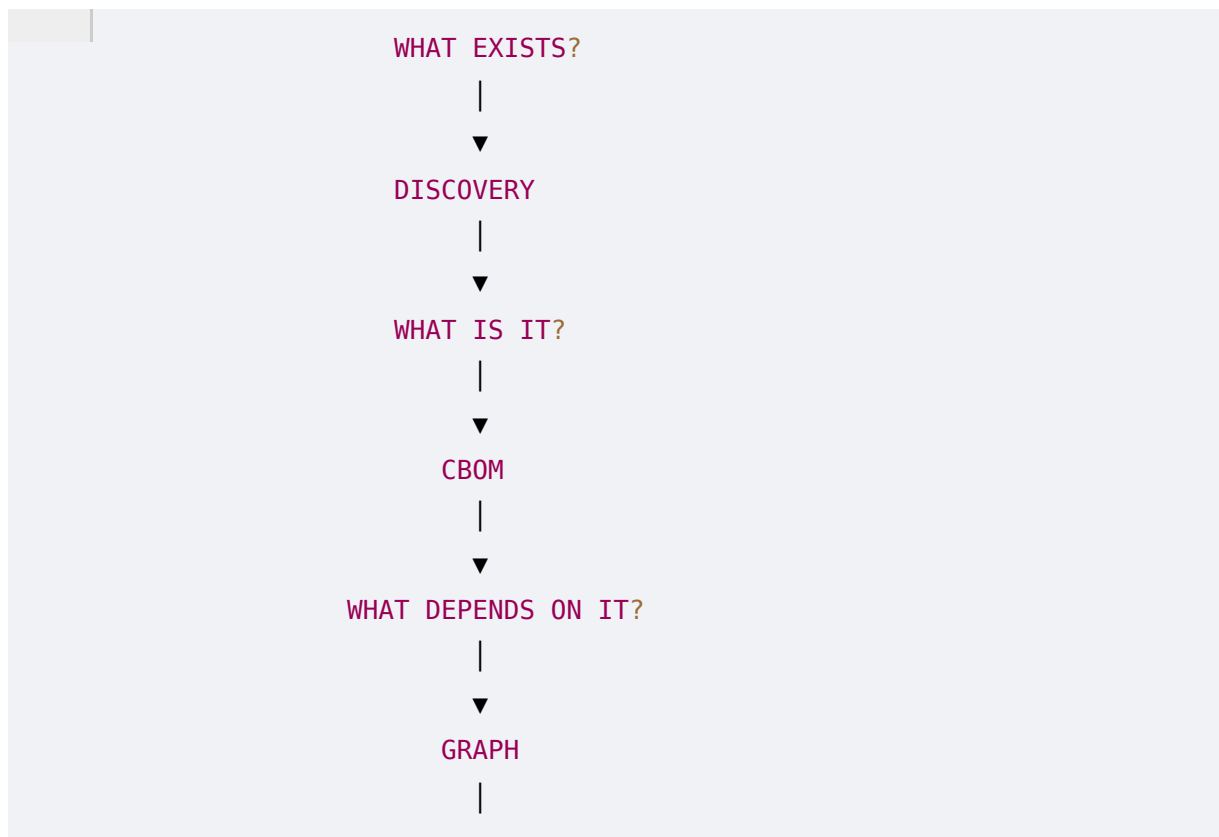
A risk score should lead to a migration decision.

5. Verification over promises

A migration isn't complete until the system rescans and verifies the new state.

148. FINAL DIFFERENTIATOR

The strongest conceptual model for the entire project is:



That is ECDAT.

149. THE BIGGER VISION

The SIH prototype is only the beginning.

The eventual platform could become:

An enterprise cryptographic digital twin.

A security team could ask:

"Show me every cryptographic dependency in our organization."

"Which critical data is exposed to quantum-vulnerable cryptography?"

"What happens if we replace this provider?"

"Which five migrations reduce the most risk?"

"Which applications are blocked by this HSM?"

"What percentage of our cryptographic estate is PQC-ready?"

"Did our migration actually remove the legacy algorithm?"

And ECDAT answers using:

Evidence

+

CBOM

+

Graph

+

Risk

+

Policy

+

AI

150. FINAL ONE-LINE VISION

ECDAT transforms an organization's invisible cryptographic footprint into a living, evidence-backed map of quantum risk and migration readiness.

151. FINAL PROJECT STATEMENT

If you need one final statement for the entire project documentation:

Enterprise Cryptographic Discovery & Analysis Tool (ECDAT) is a security-focused platform designed to discover, inventory, analyze and govern cryptographic assets across enterprise software and infrastructure. By combining multi-layer cryptographic discovery, CBOM generation, dependency-aware knowledge graphs, quantum-risk assessment, policy-driven PQC recommendations, AI-assisted analysis

and migration verification, ECDAT provides organizations with a structured path from cryptographic visibility to post-quantum readiness.

152. FINAL SUCCESS CRITERIA

The project is successful if a judge can watch the following happen:

```
Repository
  ↓
"Found RSA."
  ↓
"Here is the exact evidence."
  ↓
"These 47 applications depend on it."
  ↓
"These 8 are critical."
  ↓
"This data has long-term confidentiality requirements."
  ↓
"Therefore these assets are high priority."
  ↓
"Here is the approved migration strategy."
  ↓
"Here are the blockers."
```

At that point, you are no longer demonstrating a collection of features.

You are demonstrating a **complete security workflow**.

153. FINAL 22-PHASE ECDAT JOURNEY

The entire documentation now forms:

```
PHASE 01
Problem Understanding
  ↓
PHASE 02
Cryptographic Foundations
  ↓
PHASE 03
Enterprise Crypto Discovery
  ↓
```

PHASE 04

CBOM Architecture



PHASE 05

Source Code Analysis



PHASE 06

Binary / Container / Certificate Discovery

154. THE FINAL MESSAGE

Bro, if you guys choose **SIH26164**, don't think of the project as:

"Let's make a crypto scanner."

That's too small.

Think of it as:

"Let's build the intelligence layer that helps an enterprise understand its entire cryptographic estate and move it toward post-quantum readiness."

The scanner is only the entry point.

The real product is:

CRYPTOGRAPHIC VISIBILITY

+

DEPENDENCY INTELLIGENCE

+

QUANTUM RISK

+

PQC STRATEGY

+

MIGRATION PLANNING

+

VERIFICATION

=

ECDAT

And the fundamental loop is:

**DISCOVER → UNDERSTAND → PRIORITIZE →
MIGRATE → VERIFY**

Phase 22 complete.

That completes the **22-phase ECDAT problem-statement analysis and implementation blueprint.**